

0.103. Une lettre qui tire — le B tire aussi

Le 0.102.1, et le B riposte : toutes les 1,2 seconde, un « . » part vers la gauche, sur sa ligne. S'il touche le A, une vie de moins ; à 0 : PERDU, et START pour recommencer (le 0.84).

1. Ce qu'il faut comprendre

C'est le 0.102.1, avec une chose en plus : le B tire aussi. Toutes les 1,2 seconde, le B lance un **point** « . » vers la gauche, sur sa ligne. S'il touche le A, le A perd une **vie**. Au bout de trois : **PERDU**.

Ce qui est nouveau ici : le tir du B. Trois variables : `ex`, `ey`, sa place, et `evole`, 1 s'il est en vol. Il va **toujours à gauche** : pas besoin de sens, l'arrivée est simplement `ex - 1`.

Le lancer : `chaque(1200)`, et seulement si aucun tir du B ne vole, si le B n'est pas en colonne 0 (il n'y aurait pas de place à sa gauche), et si la case à sa gauche est vide.

Le pas, toutes les 150 ms : le même plan que pour le tir du A. On efface sa case, puis : **1.** l'arrivée est hors de l'écran (`nx >= 20` : 0 - 1 donne 255) : il disparaît. **2.** L'arrivée est **le A** (`nx == ax && ey == ay`) : touché ! Une vie de moins, un `bruit()` (le 0.83.1), et s'il n'en reste plus, PERDU. **3.** L'arrivée n'est pas vide (un X, ou un tir du A) : il s'écrase. **4.** Sinon, il y va.

Le tir du B ressemble à ceux du A : un « . », lui aussi. **Les tirs se bloquent :** un tir qui trouve un autre tir sur sa route s'y écrase (le cas « pas vide »), qu'il soit du A ou du B. On peut donc **se défendre** en tirant sur les tirs du B.

Les vies et l'état du jeu, comme au 0.84 : `vies` part de 3. `etat` vaut 0 quand on joue, 1 quand on a perdu. Toute la partie est dans `if (etat == 0) { ... }` ; dans le `else`, plus rien ne bouge, on attend **START** : il efface PERDU, remet le A et le B (PERDU les a peut-être recouverts), 3 vies, score 0, et on rejoue.

Déroulons : A en (2, 8). HAUT jusqu'en (2, 2), sur la ligne du B. Le B, en (16, 2), lance un « . » en (15, 2). Il avance : (14, 2), (13, 2)... L'arrivée devient (2, 2) : c'est le A. `vies` passe à 2, et ainsi de suite, jusqu'à 0 : PERDU.

Essaie : fais tirer le B plus souvent (`chaque(600)`), ou donne 5 vies au A.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <note>` → [0.82.2. La fonction note\(\) — jouer une note](#)
- `#include <bruit>` → [0.83.1. La fonction bruit\(\) — un bruit](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2722 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	121 o	← la console, à l'allumage – la mise en route de la console
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
placerB()	55 o	← main() – ta fonction
main()	1582 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
AdresseCase	10 o	← lire(), poser(), effacer(), texte() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() – écrit un nombre en chiffres
Reste	10 o	← placerB() – le reste d'une division
CopierTuiles	94 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Hasard	25 o	← hasard() – tire un nombre au hasard
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	384 o	← CopierTuiles – 24 lettres sur 43
Notes	120 o	← note() – la table des notes de musique
"SCORE"	5 o	← texte() – un texte gravé
"VIES"	4 o	← texte() – un texte gravé
". "	1 o	← texte() – un texte gravé
". "	1 o	← texte() – un texte gravé
". "	1 o	← texte() – un texte gravé
"PERDU"	5 o	← texte() – un texte gravé
". "	1 o	← texte() – un texte gravé
"SCORE"	5 o	← texte() – un texte gravé
"VIES"	4 o	← texte() – un texte gravé

Lettres gravées : ABCDEIOPRSUVX0123456789.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert

Fonctions du langage appelées :

texte()	x 9	227 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()	x 5	118 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
poser()	x 7	90 o sur place, dans placerB(), main() + AdresseCase (10 o)
lire()	x 6	68 o sur place, dans placerB(), main() + AdresseCase (10 o)
nombre()	x 6	60 o sur place, dans main() + EcrireNombre (33 o)
note()	x 1	56 o sur place, dans main() + Notes (120 o)
bruit()	x 1	39 o sur place, dans main()
bouton()	x 6	30 o sur place, dans main() + LireManette (37 o)
hasard()	x 2	6 o sur place, dans placerB() + Hasard (25 o)
image()	x 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	x 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

chaque_minuteur()	x 5	54 o sur place, dans main()
placerB()	x 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : le B tire aussi ----
// Le 0.102.1, et la cible B RIPOSTE : toutes les 1,2 seconde, elle lance
// un « . » vers la gauche, sur sa ligne : un tir pareil à ceux du A.
// S'il touche le A : une vie de moins.
// À 0 vie : PERDU, et START pour recommencer (le 0.84).

#include <hasard>      // tire un nombre au hasard
#include <lire>        // lit la tuile posée sur une case
#include <poser>       // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <texte>       // écrit un texte à l'écran
#include <nombre>      // écrit un nombre en chiffres
#include <chaque>      // répond 1 toutes les n millisecondes
#include <bouton>      // lit un bouton de la manette
#include <effacer>     // efface des cases, ou tout le fond
#include <note>        // joue une note
#include <bruit>        // joue un bruit
#include <reste>       // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
```

2. UN ENREGISTREMENT — TIR

```
struct Tir {
    uint8_t x, y;           // sa place
    uint8_t dx, dy;        // son sens
    uint8_t vole;          // 1 = en vol ; 0 = libre
};
```

3. UN TABLEAU — TIRS

```
Tir tirs[3];              // les trois tirs du A (le 0.102.1)
```

4. UNE VARIABLE — AX

```
uint8_t ax = 2;           // le A : sa colonne...
```

5. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...et sa ligne
```

6. UNE VARIABLE — DX

```
uint8_t dx = 1;           // le sens où regarde le A
```

7. UNE VARIABLE — DY

```
uint8_t dy = 0;
```

8. UNE VARIABLE — NX

```
uint8_t nx = 0;           // une case d'ARRIVÉE, calculée avant de bouger
```

9. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

10. UNE VARIABLE — BX

```
uint8_t bx = 16;          // la cible B
```

11. UNE VARIABLE — BY

```
uint8_t by = 2;
```

12. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

13. UNE VARIABLE — LANCE

```
uint8_t lance = 0;           // 1 = un tir vient de partir, à ce tour-ci
```

14. UNE VARIABLE — EX

```
uint8_t ex = 0;             // NOUVEAU : le tir du B, le « . » : sa colonne...
```

15. UNE VARIABLE — EY

```
uint8_t ey = 0;            // ...sa ligne...
```

16. UNE VARIABLE — EVOLE

```
uint8_t evole = 0;         // ...et 1 s'il est en vol. Il va toujours à gauche.
```

17. UNE VARIABLE — VIES

```
uint8_t vies = 3;          // NOUVEAU : les vies du A (1e 0.84)
```

18. UNE VARIABLE — ETAT

```
uint8_t etat = 0;          // L'ÉTAT DU JEU (1e 0.84) : 0 = on joue, 1 = perdu
```

19. UNE FONCTION — PLACERB()

```
void placerB() {  
    do {  
        bx = hasard() % 20;  
        by = hasard() % 17;  
    } while (lire(bx, by) != 0);  
    poser(bx, by, ALPHABET[1]);  
}
```

20. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t l = 4; l < 13; l++) {
        poser(12, l, ALPHABET[23]);
    }
    poser(ax, ay, ALPHABET[0]);
    poser(bx, by, ALPHABET[1]);
    texte(0, 17, "SCORE");
    nombre(6, 17, score);
    texte(10, 17, "VIES");           // NOUVEAU : les vies, à côté du score
    nombre(15, 17, vies);

    while (true) {
        image();

        if (etat == 0) {             // ===== ON JOUE

            // ---- le A bouge et retient où il regarde (le 0.100) ----
            if (chaque(150)) {
                nx = ax;
                ny = ay;
                if (bouton(DROITE)) {
                    nx = ax + 1;
                    dx = 1;
                    dy = 0;
                } else if (bouton(GAUCHE)) {
                    nx = ax - 1;
                    dx = -1;
                    dy = 0;
                } else if (bouton(BAS)) {
                    ny = ay + 1;
                    dx = 0;
                    dy = 1;
                } else if (bouton(HAUT)) {
                    ny = ay - 1;
                    dx = 0;
                    dy = -1;
                }
                if (nx < 20 && ny < 17) {
                    if (lire(nx, ny) == 0) {
                        effacer(ax, ay, 1);
                        ax = nx;
                        ay = ny;
                        poser(ax, ay, ALPHABET[0]);
                    }
                }
            }

            // ---- A lance un tir, au plus toutes les 250 ms (le 0.102.1) ----
            if (chaque(250)) {
                if (bouton(A)) {
                    nx = ax + dx;
                    ny = ay + dy;
                    if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
                        lance = 0;
                        for (uint8_t i = 0; i < 3; i++) {
                            if (lance == 0 && tirs[i].vole == 0) {
                                tirs[i].vole = 1;
                                tirs[i].x = nx;
                                tirs[i].y = ny;
                                tirs[i].dx = dx;
                                tirs[i].dy = dy;
                                texte(tirs[i].x, tirs[i].y, ".");
                                lance = 1;
                            }
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
  }
}

// ---- chaque tir du A avance (le 0.102.1) ----
if (chaque(100)) {
  for (uint8_t i = 0; i < 3; i++) {
    if (tirs[i].vole == 1) {
      nx = tirs[i].x + tirs[i].dx;
      ny = tirs[i].y + tirs[i].dy;
      effacer(tirs[i].x, tirs[i].y, 1);
      if (nx >= 20 || ny >= 17) {
        tirs[i].vole = 0;
      } else if (nx == bx && ny == by) {
        tirs[i].vole = 0;
        effacer(bx, by, 1);
        score = score + 1;
        nombre(6, 17, score);
        note(1, D05, 10, 12);
        placerB();
      } else if (lire(nx, ny) != 0) {
        tirs[i].vole = 0;          // un X, ou un tir (même celui du B)
      } else {
        tirs[i].x = nx;
        tirs[i].y = ny;
        texte(tirs[i].x, tirs[i].y, ".");
      }
    }
  }
}

// ---- NOUVEAU : le B lance son tir, toutes les 1200 ms ----
// Un seul à la fois (evole == 0), juste à GAUCHE du B, si la case est vide.
if (chaque(1200)) {
  if (evole == 0 && bx > 0) {
    if (lire(bx - 1, by) == 0) {
      evole = 1;
      ex = bx - 1;
      ey = by;
      texte(ex, ey, ".");
    }
  }
}

// ---- NOUVEAU : le tir du B avance vers la gauche, toutes les 150 ms ----
// Le même plan que pour le tir du A : l'arrivée d'abord, puis on décide.
if (chaque(150)) {
  if (evole == 1) {
    nx = ex - 1;          // toujours à gauche (en colonne 0 : 255)
    effacer(ex, ey, 1);
    if (nx >= 20) {
      evole = 0;          // 1. sorti par la gauche
    } else if (nx == ax && ey == ay) {
      evole = 0;          // 2. le A est touché !
      vies = vies - 1;    //   une vie de moins...
      nombre(15, 17, vies);
      bruit(8, 12);       //   ...un bruit : 8 images, volume 12 (le 0.83.1)...
      if (vies == 0) {     //   ...et plus de vie : PERDU
        etat = 1;
        texte(7, 8, "PERDU"); //   écrit au milieu de l'écran
      }
    } else if (lire(nx, ey) != 0) {
      evole = 0;          // 3. un X, ou un tir du A : il s'écrase
    } else {
      ex = nx;            // 4. une case vide : il y va
    }
  }
}

```

```

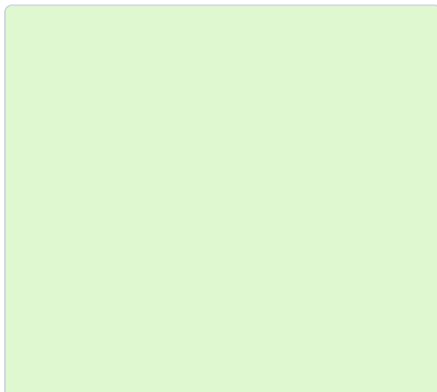
        texte(ex, ey, ".");
    }
}
}

} else { // ===== PERDU
// Plus rien ne bouge : on attend START (le 0.84).
if (bouton(START)) {
    effacer(7, 8, 5); // efface les 5 lettres de PERDU...
    poser(ax, ay, ALPHABET[0]); // ...et remet le A et le B : PERDU
    poser(bx, by, ALPHABET[1]); // les a peut-être recouverts
    vies = 3;
    score = 0;
    texte(0, 17, "SCORE"); // la ligne du bas, comme au départ
    nombre(6, 17, score);
    texte(10, 17, "VIES");
    nombre(15, 17, vies);
    etat = 0; // et on rejoue
}
}
}
}
}

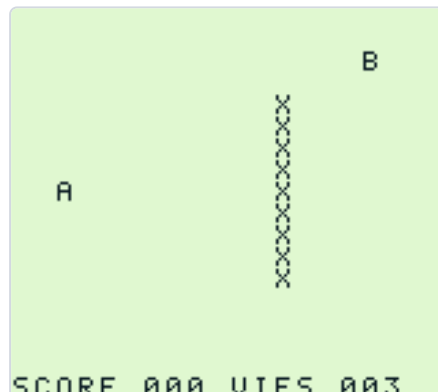
```

3. Ce qui se passe, étape par étape

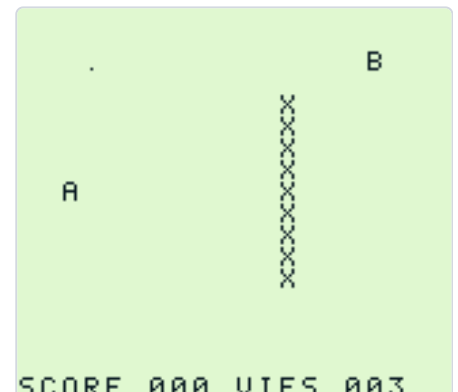
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



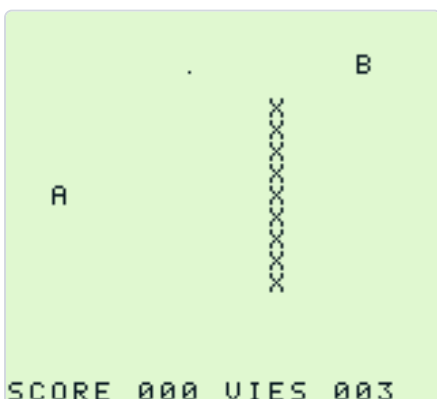
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



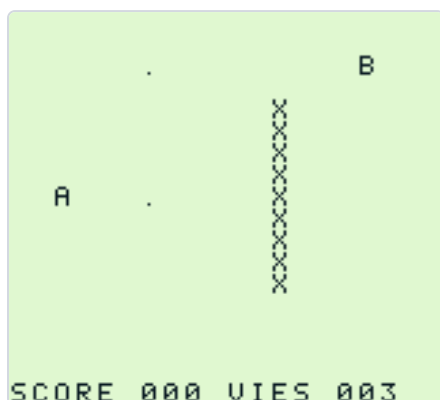
au bout de trois secondes — image 180



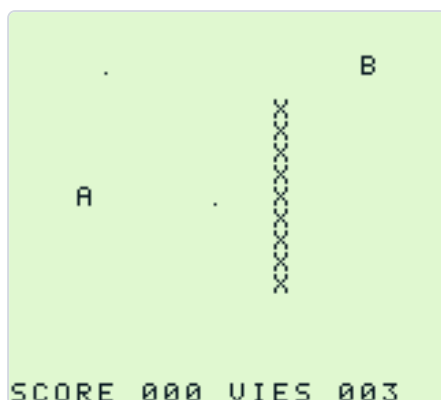
au bout de sept secondes — image 420

4. Ce que les touches changent

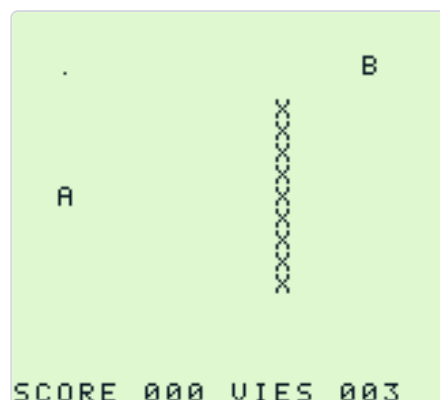
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



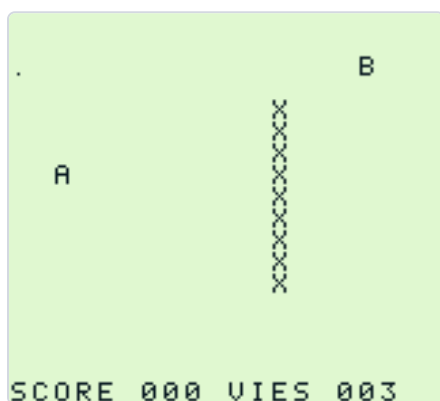
A — l'écran a changé



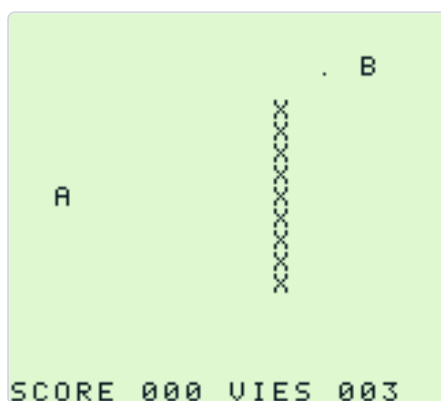
DROITE — l'écran a changé



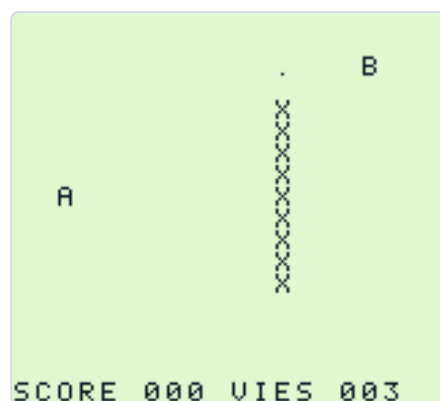
GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé



START — l'écran a changé

5. Ce qu'on doit voir

Le B lance des « . » vers la gauche. Sur sa ligne, le A est touché : les vies baissent, et à 0, PERDU s'affiche. START relance la partie.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ : VIES 003
- ✓ le B lance un « . » vers la gauche, sur sa ligne
- ✓ le A sur la ligne du B est touché : VIES 002
- ✓ à 0 vie : PERDU, au milieu de l'écran
- ✓ perdu : le A ne bouge plus
- ✓ START : PERDU effacé, 3 vies, on rejoue

Ensuite : **0.104. Une lettre qui tire — un petit jeu : les W tombent** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++